

Bare-Metal RTOS for Autonomous Vehicle Platform ARM Cortex-M3

Problem Statement

Autonomous vehicle subsystems — perception, control, and hardware monitoring — must run concurrently with deterministic timing and strict priority ordering, with no OS or runtime to rely on. RuncOS solves this by implementing a cooperative bare-metal RTOS kernel from scratch on an ARM Cortex-M3 microcontroller, targeting the constraints of embedded AV platform hardware: no heap, no stdlib, no FPU, and a fixed 20 KB RAM budget.

System Overview

Key	Value
Target Hardware	ARM Cortex-M3 QEMU lm3s6965evb 12 MHz
Tick Rate	1000 Hz (SysTick reload = 11999 cycles, 1 ms per interrupt)
RAM Budget	20 KB total task stacks in .bss zeroed by startup.s
FLASH Budget	256 KB code + .rodata + .data load image
Math System	Q16.16 fixed-point throughout — no FPU, no float, no libm
Stdlib / Runtime	None — compiled with -ffreestanding -nostdlib -nostartfiles
Number of Tasks	3 real-time tasks 6 pipeline stages per tick
Perception Cycle	20 s (0.00 → 10.00 → 0.00 triangular wave)
HW Check Interval	30 s simulated alternating pass/fail
Pullover Duration	5 s brake=1.00 steer=+1.00 (hard right) full state restore

Workflow Overview

Every 1 ms the SysTick ISR fires and calls `on_tick()`, which executes all six pipeline stages in strict priority order. Shared state is passed through volatile Q16.16 globals — no queues, no locks, no RTOS primitives needed at this stage. The

architecture is deliberately designed to scale: replacing the globals with lock-free ring buffers or CMSIS-RTOS queues requires no structural changes.

`on_tick()` pipeline — 6 stages, strict priority, 1 ms tick period:

1. PERCEPTION — triangular wave, 0.00→10.00→0.00, period 20s
2. HW CHECK — fires every 30s, triggers pullover on fail
3. CONTROL — brake + steer output
4. STATS — drift tracking
5. EVENTS — edge-detect alerts
6. PRINT — 1 Hz snapshot

`on_tick()` runs all 6 stages every 1 ms tick 1000 Hz SysTick ISR

Memory Layout

The linker script (`linker.ld`) places `.vector_table` at address `0x0000_0000` — a hard requirement on Cortex-M3 where the hardware loads MSP from word 0 and jumps to `Reset_Handler` at word 1 on power-on. The `.data` section has a split LMA/VMA: it is stored in FLASH and copied to RAM at boot by `startup.s`. The `.bss` section — which holds all task stacks — is zeroed by the same startup loop, eliminating any need for explicit `memset` calls.

FLASH 256 KB (`0x0000_0000` → `0x0004_0000`):

- `.vector_table` @ `0x0000_0000`
- `.text` (code) @ `0x0000_xxxx`
- `.rodata`
- `.data` (LMA) — load copy

RAM 20 KB (`0x2000_0000` → `0x2000_5000`):

- `.data` (VMA) @ `0x2000_0000`
- `.bss` (zeroed) — task stacks
- (stack) @ `0x2000_5000`
- (free)

`startup.s` copies `.data` from LMA to VMA and zeroes `.bss` before `kernel_main()`

Linker script: `linker.ld` startup: `startup.s` (ARM Thumb assembly)

Boot Sequence

The boot sequence is implemented entirely in ARM Thumb assembly (startup.s). After reset, Reset_Handler copies .data from FLASH to RAM, zeroes .bss, then branches to kernel_main(). The vector table wires SysTick_Handler (entry 15) to a Thumb-mode trampoline that calls on_tick() and returns cleanly via pop {pc}.

```
Reset_Handler:                ; ARM Thumb assembly (startup.s)
    ldr r0,=_data_load_start    ; source:  .data in FLASH (LMA)
    ldr r1,=_data_start        ; destination: .data in RAM (VMA)
    ldr r2,=_data_end
.Lcopy: cmp r1,r2 | bge .Lbss    ; copy loop until _data_end
    ldr r3,[r0],#4 | str r3,[r1],#4 | b .Lcopy
.Lbss:  ldr r0,=_bss_start | ldr r1,=_bss_end | mov r2,#0
.Lzero: cmp r0,r1 | bge .Ldone    ; zero loop until _bss_end
    str r2,[r0],#4 | b .Lzero
.Ldone: bl kernel_main           ; jump to C++ kernel
```

Task Design

Three real-time tasks run cooperatively inside the SysTick ISR. Each task is a plain C++ function with no dynamic allocation. All inter-task communication is through volatile Q16.16 globals — correct because all tasks run sequentially within a single ISR context and the compiler cannot cache volatile values across calls.

Data dependency graph: perception feeds control, hw_check feeds control, outputs are brake + steer.

- PERCEPTION → g_distance → CONTROL → OUTPUTS (brake 0.00–1.00, steer –1.0–+1.0, 1ms latency)
- HW CHECK → g_hw_ok → CONTROL
- HW fault → brake=1.00, steer=+1.00 (hard right)

All shared state volatile Q16.16 no floats ISR-safe zero stdlib

Task 1 — Perception (highest priority)

Generates a smooth triangular distance wave 0.00 → 10.00 → 0.00 over a 20-second period using the tick counter modulo 20,000. All arithmetic is kept within uint32_t range (verified: max intermediate = 9,999,000 — well under 4.29B). No division inside the hot path — the tick index directly maps to a centiunits value.

```
// Perception — Q16.16, no float, no overflow (verified)
uint32_t phase    = g_ticks % 20000;           // 0..19999
uint32_t half     = (phase < 10000) ? phase     // going up
                  : (19999 - phase);           // going down
```

```
uint32_t centiunits = (half * 1000u) / 10000u; // 0..1000
g_distance      = (centiunits * 65536u)/100u; // → Q16
```

Task 2 — HW Check (medium priority)

Fires exactly every 30,000 ticks (30 s) using tick modulo — no decrementing counter, so there is zero drift regardless of tick rate. The live countdown display (`g_hw_countdown`) is computed as $(\text{remaining_ticks} \ll 16) / 1000$, keeping the result in Q16 seconds. On a simulated failure, the pullover state machine arms a 5-second window: `g_pullover_end = g_ticks + 5000`.

Task 3 — Control (runs every tick, depends on tasks 1 + 2)

Reads `g_distance` and `g_hw_ok`, writes `g_brake` and `g_steer`. In normal mode, brake is the inverse-linear function of distance and steer is $\text{brake} \times 0.30$ (gentle right avoidance). During pullover, both are locked: `brake = 1.00`, `steer = +1.00` (hard right). Q16 multiply uses the standard $(a \times b) \gg 16$ pattern; all intermediates fit in `uint32_t`.

```
// Control — normal mode
q16_t inv = Q16(10) - g_distance; // 0..Q16(10)
g_brake   = inv / 10u;             // Q16 ÷ 10 → 0.00..1.00
g_steer   = (g_brake * 19661u) >> 16; // × 0.30 → 0.00..0.30
```

```
// Pullover mode (hw fault)
g_brake = 65536u; // 1.00
g_steer = 65536u; // +1.00 hard right
```

Control Waveforms

The distance and brake signals are exact mathematical inverses over each 20-second cycle. The dashed threshold line at `brake = 0.70` marks where the **BRAKING HARD** event fires. The event system is edge-detected — it fires once on the rising edge and once on the falling edge, never on every tick.

SysTick 1000 Hz SYSTICK_RELOAD = 11999 Q16.16 fixed-point math no FPU

Waveform description: Distance (green, 0–10) forms a triangle wave rising from 0 to 10 at $t=10\text{s}$ then falling back to 0 at $t=20\text{s}$. Brake (red, 0–1.00) is the exact inverse. Orange dashed threshold at `brake=0.70` marks the BRAKING HARD zone (`distance < 3.00`).

Event System & Timeline

The event queue is an 8-slot fixed-size circular buffer of 48-byte strings. Events are pushed by the ISR and flushed once per second during the print stage. Because all operations happen inside a single ISR there is no race condition and no need for critical sections. Four event types are defined:

Event	Description
BRAKING HARD	Fires when brake crosses 0.70 (distance < 3.00). Clears when brake drops back below threshold.
AVOIDANCE STEER	Fires when steer crosses 0.20. In normal mode this is always coupled with braking hard.
HW CHECK FIRED	Fires at every 30s boundary — reports PASS or FAIL. Failure immediately arms the pullover state machine.
PULLOVER START/END	PULLING OVER fires when hw_ok=0 is set. PULLOVER COMPLETE fires when g_ticks >= g_pullover_end.

Event timeline over 40s:

- t=0s: BRAKING HARD (brake > 0.70)
- t=3s: brake released
- t=17s: BRAKING HARD again
- t=20s: brake released
- t=30s: HW FAIL → PULLOVER (5s window, orange zone)
- t=35s: RESUME normal

Events edge-detected in ISR queued to print buffer flushed once per second

Performance & Accuracy

The tick accuracy system counts actual ticks per second and computes cumulative drift relative to the expected TICK_RATE_HZ. On QEMU the emulator is cycle-accurate so drift is always exactly 0 — this validates the math. On physical Cortex-M3 silicon the internal RC oscillator has ±1% tolerance (~±10 ticks/s at 1 kHz), making the drift counter useful for diagnosing clock quality without external test equipment.

Metric	Value
--------	-------

Tick Rate	1000 Hz SYSTICK_RELOAD_VAL = 11999 12,000 cycles per tick
ISR Overhead	~dozens of cycles per non-print tick UART only on print ticks (1/1000)
Drift (QEMU)	0 ticks cumulative — emulator is cycle-accurate
Drift (real silicon)	±1-2 ticks/s expected from RC oscillator tolerance (±1%)
Worst-case tick	Print tick: UART spin-loop for snapshot output (~300–400 chars @ 115200 baud ≈ 35 ms on real HW — acceptable: next tick not missed since ISR returns first)
Scaling	TICK_RATE_HZ is the single constant that rescales all timings — perception cycle, HW check interval, print interval, pullover duration

Sample Output

```
=====
t=17s | ticks=1000 drift=+0 | mode=NORMAL
=====
PERCEPTION dist=3.00 / 10.00
CONTROL brake=0.70 steer=0.21
HW CHECK ok=YES next_in=13.00s
=====
>> BRAKING HARD -- brake > 0.70
>> AVOIDANCE STEER -- steer > 0.20
=====

=====
t=30s | ticks=1000 drift=+0 | mode=PULLOVER
=====
PERCEPTION dist=9.98 / 10.00
CONTROL brake=1.00 steer=+1.00
HW CHECK ok=NO next_in=30.00s
=====
>> HW CHECK FAILED -- initiating pullover
>> PULLING OVER -- brake=1.00 steer=+1.00 (hard right)
=====
```

Build & Run

```
# Compile (ARM bare-metal, no stdlib)
arm-none-eabi-g++ -mcpu=cortex-m3 -mthumb -ffreestanding \
  -nostdlib -nostartfiles -T linker.ld \
  startup.s kernel.cpp -o kernel.elf

# Run in QEMU (Ctrl+A then X to quit)
qemu-system-arm -machine lm3s6965evb -kernel kernel.elf -nographic
```

Conclusion

RuncOS demonstrates a complete bare-metal RTOS design from linker script to application logic, built entirely without OS, stdlib, or runtime support. The three-task pipeline runs at 1000 Hz with 1 ms tick resolution, deterministic priority ordering, ISR-safe event dispatch, and autonomous hardware fault recovery — all within 20 KB of RAM using Q16.16 fixed-point arithmetic on a Cortex-M3 with no FPU. The architecture is designed to scale directly to multi-product embedded platforms: a single constant change to `TICK_RATE_HZ` rescales all timing system-wide, and the task structure maps cleanly onto CMSIS-RTOS or FreeRTOS task primitives for future porting.